Safe Robot Navigation in Dynamic Pedestrian Environments

Technical Analysis of a Multi-Algorithm GPU-Accelerated Benchmark

Detailed software, algorithmic, mathematical, and reproducibility analysis

This report provides an implementation-faithful analysis of the supplied Python program. It covers the computational pipeline, mathematical formulation, algorithmic structure, GPU strategy, validation logic, benchmark design, and packaging workflow. Figures and experimental result visualizations are outside the scope of this report.

Contents

Abstract

The supplied Python program implements a research-oriented benchmark for safe robot navigation in a two-dimensional environment containing static and harmonically moving pedestrian obstacles. It combines a cubic-spline trajectory representation with several planning families: D* Lite, conventional particle swarm optimization (PSO), linear-decreasing-weight PSO (LDW-PSO), a D*-PSO hybrid, artificial bee colony optimization (ABC), a PSO-ABC hybrid, spider monkey optimization (SMO), and ant colony optimization (ACO). The program forms a self-contained benchmarking pipeline: it generates deterministic synthetic scenarios, loads an environment model, performs collision-aware spatiotemporal path evaluation, runs each planner under controlled random seeds, validates the resulting trajectories, exports machine-readable metrics, generates optional plots, writes an audit document, and packages the outputs.

A central design feature is the separation of optimization variables from trajectory geometry. The metaheuristic planners search over a small set of two-dimensional via points, and a parametric spline converts them into a dense path. A shared GPU cost evaluator measures path length, a discrete squared-curvature surrogate, minimum clearance, and collision count. These quantities are combined in a scalar objective with collision and low-clearance penalties. D* Lite and ACO operate on a discrete occupancy grid, whereas the other planners optimize spline control points.

The analysis also highlights implementation details that matter when interpreting the benchmark. The dataset generator creates five scenarios by default, but the benchmark driver uses only the first scenario for planning. GPU acceleration is substantial for tensor-based geometry and cost computation, while spline generation and several search loops remain CPU-oriented or involve CPU–GPU transfers. These details do not invalidate the program; they clarify what the benchmark measures and how its computational claims should be understood.

1 Introduction

The project addresses a standard robotics planning problem: a robot must move from a fixed start point to a fixed goal within a bounded planar workspace while avoiding obstacles that may move over time. The code is framed as a benchmark suite rather than as a single planner. Its purpose is not only to generate a trajectory, but to provide a shared environment, path representation, cost function, validation rules, and execution instrumentation for comparing several planning strategies.

The program defines a square workspace with width and height 10 units and a grid resolution of 1 unit. The robot starts at $(0, 0)$ and targets $(6, 7)$. Eight planning configurations are benchmarked: D*, PSO, LDW-PSO, D*-PSO, ABC, PSO-ABC, SMO, and ACO. The opening documentation describes the D* implementation as an incremental replanner, while the hybrid and swarm-colony methods are presented as alternatives for global search and path refinement.

From a software-engineering perspective, the program is deliberately self-contained. It includes configuration, reproducibility controls, synthetic data generation, environment construction, planning classes, validation helpers, metric export, plotting, audit-report creation, and submission packaging. This makes it suitable as a reproducible assignment or portfolio artifact because most of the project lifecycle can be traced without an external application framework.

This report focuses on the technical and methodological analysis of the implementation. It does not present performance results, figures, or benchmark outcomes. Conclusions directly

supported by the code are stated as implementation facts; interpretations inferred from the code structure are presented as interpretations rather than documented author intentions.

2 Project Overview

2.1 End-to-end computational pipeline

The program follows a clear sequence. Runtime configuration selects the execution device and fixes the random seeds. A deterministic synthetic dataset is generated for several scenarios, with each obstacle represented by a base location, radius, motion amplitudes, angular frequency, and phase shift. The environment then loads one scenario and constructs GPU tensors for obstacle parameters and occupancy-grid calculations.

The planners solve the same navigation instance. Discrete planners build paths directly on the occupancy grid, whereas continuous planners encode trajectories with a small set of via points and pass them through a spline interpolator. Every candidate is evaluated with the same path metrics. The benchmark driver repeats each algorithm with several random seeds, synchronizes CUDA for runtime measurements, records memory and timing, validates geometry and endpoints, and exports summary tables.

Conceptually, the pipeline can be represented without relying on a diagram as the following chain:

Input configuration → synthetic pedestrian trajectories → environment model → planner-specific search → spline or grid path → shared spatiotemporal cost → validation → metric export and packaging.

2.2 Core design principle

The main unifying idea is a common evaluation model across different search strategies. D* Lite and ACO work in a discrete state space, whereas the PSO, ABC, and SMO families work in a continuous control-point space. Once a path is produced, the same geometric and collision metrics can be applied to it.

The program therefore separates *search representation* from *trajectory evaluation*. This is useful for benchmarking because it reduces the risk of judging each algorithm with a different path-quality definition.

3 Technical Foundations

3.1 Dynamic obstacle kinematics

Each obstacle is modeled by the `PedestrianObstacle` data class. Static obstacles retain a fixed base position. Dynamic obstacles follow independent harmonic motions. For obstacle j and time t, the source implements

$$x_j(t) = x_{j,0} + A_{x,j} \sin(\omega_j t + \phi_j), \tag{1}$$

$$y_j(t) = y_{j,0} + A_{y,j} \cos(\omega_j t + \phi_j). \tag{2}$$

The position is then clipped to $[0.5, W - 0.5]$ in x and $[0.5, H - 0.5]$ in y. This keeps obstacle centers away from the workspace boundary. The clipping is a kinematic constraint imposed by the implementation, not a physical model of pedestrian behavior.

3.2 Occupancy grids

The environment discretizes the continuous workspace into $N_x \times N_y$ cells. The source places a representative point at the center of each cell and marks a cell occupied if its center lies within an obstacle radius plus a safety margin. The CPU fallback explicitly uses Euclidean distance. The GPU path computes the same idea with tensor broadcasting.

If a cell center is $c = (x_c, y_c)$ and obstacle j has center $o_j(t)$ and radius r_j, the occupancy test is

$$\|c - o_j(t)\|_2 \leq r_j + m, \tag{3}$$

where m is the configurable safety margin, set to 0.4 by default.

3.3 Spline-based trajectory representation

Continuous planners do not directly optimize every sampled trajectory point. Instead, they optimize K two-dimensional via points. With start s, via points $v_1, \ldots, v_K$, and goal g, the control-point sequence is

$$P = [s, v_1, \ldots, v_K, g]. \tag{4}$$

For at least three control points, the implementation attempts a parametric B-spline using SciPy's `splprep` and `splev`; otherwise, it uses linear interpolation. A fallback piecewise interpolation path is also used when spline fitting raises a numerical or argument-related exception. The resulting samples are clipped to the workspace, while the first and last samples are explicitly reset to the exact start and goal coordinates.

3.4 Path length

For a sampled path $p_0, \ldots, p_M$, the source uses the Euclidean length of each segment:

$$L = \sum_{i=0}^{M-1} \|p_{i+1} - p_i\|_2. \tag{5}$$

This is the standard polyline approximation of continuous arc length.

3.5 Discrete smoothness / squared-curvature surrogate

The program computes a heading angle for each segment,

$$\theta_i = \operatorname{atan2}(\Delta y_i, \Delta x_i), \tag{6}$$

then unwraps successive angle differences into $(-\pi, \pi]$. With Δs_i approximated by the mean length of the neighboring segments plus a small regularizer ε, the reported smoothness is

$$S = \sum_i \frac{(\Delta\theta_i)^2}{\Delta s_i + \varepsilon}. \tag{7}$$

This expression is a discrete approximation motivated by the continuous bending-energy quantity $\int \kappa^2 \, ds$. It is more meaningful geometrically than simply summing squared heading changes because shorter segments receive a larger curvature-like contribution.

3.6 Spatiotemporal obstacle clearance

The evaluation associates a travel time with each sampled path point. Given robot speed v and segment lengths ℓ_i, the elapsed time after segment i is approximated by

$$t_i = t_0 + \sum_{k=0}^{i-1} \frac{\ell_k}{v}. \tag{8}$$

Obstacle positions are therefore evaluated at different times along the same candidate trajectory. For point p_i and obstacle j, the signed clearance is

$$d_{ij} = \|p_i - o_j(t_i)\|_2 - r_j. \tag{9}$$

The minimum clearance is the minimum of all such values. For reporting, the source clips this minimum at zero, so negative values are not exposed as negative clearance in the public metric.

Collision count is computed separately as the number of point-obstacle evaluations satisfying $d_{ij} \leq 0.05$. Thus, the collision metric is not a Boolean path-level collision indicator; it is a count over sampled point-obstacle pairs.

4 Data and Input Processing

4.1 Synthetic benchmark generation

The dataset generator creates deterministic synthetic scenarios. Each scenario contains three static circular obstacles and five dynamic obstacles. Static parameters are fixed, while dynamic obstacles receive distinct amplitudes, frequencies, and phases. The start and goal are the same across scenarios, and metadata stores the obstacle dictionaries in JSON format.

For each scenario and each time step, the code evaluates every obstacle and appends a record containing scenario identifier, time step, obstacle identifier, dynamic/static flag, position, and radius. These records are exported to CSV. Scenario-level metadata are exported to JSON.

The generator resets NumPy and Python's standard random module with a scenario-specific offset of the global seed. This is a reproducibility mechanism, although the generated obstacle specifications shown in the source are deterministic even without using scenario-local randomness. The use of scenario-specific seeds nevertheless makes the intent of repeatable scenario construction explicit.

4.2 Important benchmark-scope observation

The function defaults to five scenarios and 100 time steps. However, the benchmark driver subsequently constructs one environment and loads only `scenario_1`. Therefore, the five-

scenario generator and the one-scenario planner execution are not equivalent concepts. The code produces a multi-scenario dataset artifact, but the planner benchmark as written evaluates only the first scenario.

This distinction matters for scientific reporting. The benchmark should not be described as covering all five scenarios unless the driver is modified to loop over them. The accurate description is that the infrastructure *generates* five scenarios by default, while the planning experiment *runs on scenario 1*.

5 System Architecture and Code Organization

5.1 Runtime and reproducibility helpers

The first group of utilities establishes a small runtime layer. `_set_active_progress`, `_clear_active_progress` and `_progress_update` provide a global bridge between the benchmark progress bar and deep planner loops. `_torch_float32` centralizes tensor creation on the selected device. `_gpu_scalar_to_float` explicitly transfers scalar tensors to the CPU before converting them to Python numbers. `_cuda_synchronize` is used to make GPU timing more meaningful, while `_reset_peak_memory` and `_peak_memory_kb` support memory instrumentation.

The source selects CUDA whenever PyTorch reports a CUDA device, otherwise it falls back to CPU. Random states are initialized for Python, NumPy, and PyTorch, and CUDA receives a seed when available.

5.2 Environment layer

The `NavigationEnvironment` class is the central shared state object. It stores workspace dimensions, grid dimensions, start and goal coordinates, obstacle objects, and cached GPU tensors. Loading a scenario converts per-obstacle attributes into device-resident tensors, then creates a GPU mesh of cell-center coordinates.

This object exposes two conceptually different APIs: a Python/NumPy-facing API for discrete or final computations, and a GPU-oriented tensor API for repeated geometric evaluation.

5.3 Planner layer

The planner layer contains one class for each optimization strategy. D* Lite uses dictionaries and a heap for its incremental shortest-path machinery. The continuous metaheuristics store populations of vectors with dimension $2K$, corresponding to K two-dimensional via points. ACO instead creates a four-dimensional pheromone tensor indexed by directed grid transitions.

5.4 Validation, export, and packaging layer

The validation helper tests numerical finiteness, shape, exact endpoint agreement, and workspace bounds. The benchmark driver stores per-run validation records and aggregate statistics. The source also exports representative path coordinates, dataset summaries, a run summary JSON file, an audit Markdown file, and a zip archive containing data and generated artifacts.

6 Detailed Methodology

6.1 D* Lite implementation

The D* planner uses the standard D* Lite state variables $g(s)$ and $rhs(s)$, together with a lexicographically ordered key. The key implemented by `_calculate_key` is

$$k(s) = (\min\{g(s), rhs(s)\} + h(s_{start}, s) + k_m,\ \min\{g(s), rhs(s)\})\,. \tag{10}$$

The heuristic is the Euclidean distance between two grid nodes. The edge cost is Euclidean grid distance unless the target node is occupied, in which case the edge is assigned infinity.

The `_update_vertex` routine recomputes rhs using the best successor. The shortest-path loop then applies the two fundamental D* Lite cases: if $g > rhs$, the state becomes locally consistent by setting $g = rhs$; otherwise g is invalidated and neighboring states are reconsidered.

When planning, the current occupancy grid is built at the supplied time. The code initializes the goal with $rhs(goal) = 0$, pushes it onto the priority queue, and runs the shortest-path computation. Path reconstruction then follows the neighbor with the smallest value of $g(n) + c(current, n)$.

There are two deliberate safeguards. If start or goal lies in an occupied cell, the planner returns a two-point fallback path. If no finite route is found, the same fallback is returned. These safeguards preserve a valid array structure, but they should not be interpreted as evidence that the fallback is collision-free.

The source calls this class a D* Lite incremental replanner, but the public `plan` method resets g, rhs, the heap, and related state for every call. Therefore, the implementation contains the core D* Lite mechanism but does not exploit repeated calls as a persistent incremental replanning session. In the benchmark configuration, it behaves more like a fresh D* Lite solve for each evaluated planning instance.

6.2 Particle Swarm Optimization

Each PSO particle is represented by a vector $x_i \in \mathbb{R}^{2K}$. Its velocity update follows the conventional form

$$v_i^{(t+1)} = w_t v_i^{(t)} + c_1 r_{1,i}^{(t)} \odot (p_i - x_i^{(t)}) + c_2 r_{2,i}^{(t)} \odot (g - x_i^{(t)}), \tag{11}$$

followed by velocity clipping and position clipping to the workspace. The source evaluates particles in a shared batched cost routine and tracks personal and global best states.

The initial population is centered on a linear interpolation between the start and goal, with Gaussian perturbations. When a D* guide is provided, via points sampled from that guide replace the purely straight-line initialization.

6.3 LDW-PSO

LDW-PSO is the same PSO implementation with a linearly decreasing inertia weight. The source implements

$$w_t = w_{\max} - (w_{\max} - w_{\min}) \frac{t}{I}, \tag{12}$$

where I is the configured iteration count. The intent is a common exploration-to-exploitation schedule: earlier iterations use stronger inertia, while later iterations reduce the inertial component.

6.4 D*-PSO hybrid

The hybrid first calls D* Lite to obtain a discrete global guide and then passes that guide into LDW-PSO. The PSO initialization selects K interior guide points using evenly spaced indices. This combines a coarse obstacle-aware search with continuous spline refinement.

The source returns three objects internally: the smoothed hybrid path, convergence history, and the raw D* path. The benchmark wrapper intentionally retains only the first two for metric comparison.

6.5 Artificial Bee Colony

ABC maintains a population of food-source candidates and a trial counter. In the employed-bee phase, a candidate is generated by selecting another source and adding a scaled difference. The onlooker phase samples sources with probability proportional to

$$p_i = \frac{1/(1 + J_i)}{\sum_j 1/(1 + J_j)}. \tag{13}$$

The scout phase replaces solutions whose trial counter reaches the limit with a new perturbation around the straight-line base. The algorithm is therefore a population-based stochastic search with explicit abandonment and reinitialization.

A notable implementation detail is that individual candidates are often evaluated through single-element batches. Consequently, the cost calculation itself is GPU-capable, but the overall ABC control flow remains dominated by Python loops and repeated candidate construction.

6.6 PSO-ABC hybrid

The PSO-ABC hybrid combines a PSO velocity update with an additional differential perturbation inspired by ABC-style neighborhood exploration. For individual i, the proposed position adds a PSO-driven displacement and a differential term based on a randomly selected partner. Poorly improving individuals accumulate trials and are reinitialized when the limit is reached.

The method therefore attempts to combine directed attraction toward personal/global best states with diversity-promoting pairwise exploration. The code does not claim a formal convergence theorem; its behavior is that of a heuristic hybrid optimizer.

6.7 Spider Monkey Optimization

The SMO implementation partitions the population into groups. During the local leader phase, each group identifies its best current member and generates candidate positions that move toward the local leader while also using a randomly selected group member to perturb the current solution. A global leader phase then moves each monkey toward the best global solution with an analogous differential term.

The implementation uses a fixed number of groups and does not include every variation found in the broader SMO literature. It is therefore more accurate to describe it as SMO-inspired, with explicit local- and global-leader phases, rather than as an exact implementation of every canonical SMO formulation.

6.8 Ant Colony Optimization

ACO operates directly on the discrete grid. Pheromone is stored for directed node-to-node transitions. At a given node, the candidate transition probability is proportional to

$$P_{i\to j} = \frac{\tau_{ij}^{\alpha}\eta_{ij}^{\beta}}{\sum_{k\in\mathcal{N}(i)}\tau_{ik}^{\alpha}\eta_{ik}^{\beta}}, \tag{14}$$

where heuristic desirability is based on reciprocal distance to the goal,

$$\eta_{ij} = \frac{1}{d(j,g) + 10^{-3}}. \tag{15}$$

After ants complete valid paths, pheromone evaporates according to

$$\tau \leftarrow (1-\rho)\tau, \tag{16}$$

and each successful path deposits an amount proportional to $1/L$. The implementation also prevents an ant from revisiting an already visited node and limits each walk to at most $|V|$ steps.

The pheromone tensor has shape (N_x, N_y, N_x, N_y), which is effectively $O(|V|^2)$ storage for a general grid representation. At the configured 10-by-10 grid this is modest, but the scaling implication is important for larger workspaces.

7 Mathematical Formulation of the Shared Objective

The continuous planners use a shared scalar objective combining geometry and safety terms. In source-faithful notation, the objective has the form

$$J = L + \lambda_S S + \lambda_C C + P_{clr}, \tag{17}$$

with $\lambda_S = 0.1$ and $\lambda_C = 500$ in the metaheuristic implementations.

The low-clearance penalty is piecewise:

$$P_{clr}(c) = \begin{cases} \dfrac{50}{c+0.1}, & c < 0.3, \\ 0, & c \geq 0.3, \end{cases} \tag{18}$$

where c is the minimum signed clearance before its public reporting clamp. This is an important implementation detail: the penalty is not mathematically constrained to be positive for arbitrary negative c. The large collision term usually dominates in collision cases, but the formula itself can become singular or negative for sufficiently negative clearance. A technically cautious report

should record this as a numerical consideration rather than silently rewriting the source into a different objective.

The shared evaluator is designed to distinguish three different safety notions: the occupancy grid's inflated-cell rule, the continuous minimum-clearance metric, and the sampled collision threshold. These are related but not identical. Consequently, a cell marked occupied under a 0.4 safety margin is not exactly equivalent to a continuous path point satisfying the 0.05 collision threshold.

8 Optimization and Learning Procedure

None of the planners performs gradient-based learning. The term *learning*, if used at all, therefore refers only to optimization in a broad sense. The stochastic optimizers rely on direct function evaluation, comparison, and population updates.

The PSO family maintains personal and global best states. ABC uses employed, onlooker, and scout phases. PSO-ABC combines PSO and differential perturbations. SMO alternates local and global leader dynamics. ACO updates pheromone values based on successful paths. D* Lite is fundamentally different: it is a graph-search replanning algorithm rather than a stochastic optimizer.

The code fixes seeds before each benchmark run using $42 + 10r$, where r is the run index. This ensures repeatable random streams under the same software and hardware configuration, but it should not be interpreted as proof of fully deterministic GPU execution across all environments.

9 Implementation Details

9.1 GPU strategy

The program uses PyTorch as its common tensor backend. Obstacle parameters are moved to the active device when the environment is loaded, and grid coordinates are precomputed with `torch.meshgrid`. The main GPU benefit comes from repeated tensor operations such as obstacle-position evaluation, path-segment length calculations, angle calculations, distance matrices, collision counting, and population-level objective evaluation.

The introductory claim that all algorithms are vectorized should be interpreted carefully. Several planners still construct splines in NumPy/SciPy and loop over individuals in Python. ABC, PSO-ABC, and SMO often evaluate one candidate at a time. ACO also performs substantial path construction and pheromone deposition through Python loops, although probability calculations and pheromone storage use tensors. The program is therefore better described as *GPU-accelerated with vectorized cost evaluation and tensor kernels* rather than as a fully GPU-native end-to-end implementation.

9.2 GPU timing discipline

Because CUDA kernels are asynchronous, the benchmark synchronizes CUDA before starting and after finishing each timed planner invocation. This is a sound measurement practice: without synchronization, a CPU wall-clock interval can undercount outstanding GPU work.

9.3 Memory measurement

CPU allocation tracking uses `tracemalloc`. GPU peak allocation is read from PyTorch's CUDA memory statistics. The helper returns the larger of the CPU and GPU peak values after converting both to kilobytes. This provides a coarse cross-domain indicator, but it is not a complete systems-level memory profile: allocator behavior, library workspaces, and native allocations outside Python can differ from what `tracemalloc` observes.

9.4 Progress reporting

The benchmark creates a `tqdm` progress bar per algorithm and run. Deep planner loops invoke the global progress callback. The progress total is heuristic: D* uses the grid area, the hybrid uses the grid area plus 60, ACO uses 40, and the metaheuristics default to 60. These totals represent iteration-style progress rather than an exact count of low-level operations.

10 Execution Workflow

The main entry point is `run_full_pipeline(num_scenarios=5, time_steps=100, num_runs=5)`.

The runtime first prints device and PyTorch version information, then generates the synthetic dataset and metadata. An environment is created and loaded with `scenario_1`, after which the eight benchmark configurations are registered in a dictionary.

For every algorithm, the source executes five runs. Each run receives a deterministic seed derived from the base seed, a progress bar is created, memory statistics are reset, CUDA is synchronized, and wall time is recorded. The planner runs, after which CUDA is synchronized again. The returned path is validated and evaluated with the common cost routine.

After all runs for an algorithm are complete, the code computes means and standard deviations for runtime and path length, along with means for peak memory, smoothness, minimum clearance, and collision count. It also stores theoretical time and space complexity strings. These values are written to `computation_costs_and_metrics.csv`.

Per-run validation records are written to `path_validation_results.csv`. Representative path coordinates are written to `planned_paths.csv`. An aggregated obstacle dataset summary is written to `dataset_summary.csv`. A JSON run summary contains device information, dataset size, benchmark parameters, paths to generated artifacts, and a Boolean aggregate validity flag.

The final stage writes an audit Markdown file and creates a zip archive containing selected dataset, table, figure-directory, script, notebook, and audit artifacts. In a Colab runtime, the source attempts to trigger a browser download of the archive.

11 Design Decisions and Rationale

11.1 Common trajectory abstraction

Using a common spline representation for the continuous metaheuristics reduces dimensionality. Instead of optimizing all 50 or 60 sampled path coordinates directly, each optimizer controls only K via points. For $K = 4$, the search dimension is 8. This makes population-based search more manageable than direct optimization over every sampled point.

11.2 Shared cost evaluation

Centralizing the path metrics in `NavigationEnvironment.calculate_path_cost_gpu` prevents each optimizer from implementing its own collision and smoothness logic. This is particularly valuable in a benchmark because inconsistencies in evaluation criteria can otherwise dominate algorithm comparisons.

11.3 Guide-based initialization in D*-PSO

The hybrid uses D* as a coarse guide rather than as a final trajectory. This separation is sensible: D* supplies obstacle-aware structure on the grid, while PSO searches a continuous control-point space and produces a spline path.

11.4 Explicit endpoint enforcement

The spline helper restores the first and last points after interpolation. This guarantees exact endpoint values even if the interpolation procedure introduces small numerical differences. The validation function then checks these endpoint errors against a tolerance of 10^{-6}.

12 Computational Considerations

12.1 Complexity of discrete search

The source records the standard D* Lite-style graph-search complexity as

$$O((|V| + |E|) \log |V|) \tag{19}$$

with $O(|V|)$ state storage. For an eight-neighbor grid, $|E|$ is proportional to $|V|$, so the time bound is near-linear up to the logarithmic factor in the number of grid nodes for a single solve.

For the metaheuristics, the code records $O(IND)$, where I is the number of iterations, N is population size, and $D = 2K$ is search dimension. This is an appropriate high-level abstraction for the population update stage, but it does not capture all costs of spline generation and shared trajectory evaluation. Those operations depend on the number of sampled path points and obstacle count as well.

ACO is recorded as $O(IM|V|)$ time and $O(|V|^2)$ space, with M ants. The storage expression follows the four-dimensional pheromone tensor used by the source. For larger grids, this can become the dominant memory concern.

12.2 Actual versus nominal GPU complexity

Asymptotic complexity and accelerator utilization are different dimensions. A tensorized $O(NM)$ distance calculation may execute efficiently on a GPU, while a smaller algorithm can still spend most of its time in Python loops, SciPy spline generation, or host-device transfers. The benchmark therefore reflects both algorithmic work and the implementation's heterogeneous execution strategy.

13 Reproducibility and Reliability

The program includes several concrete reproducibility mechanisms. A global seed is defined, Python/NumPy/PyTorch are seeded, and each benchmark run uses a deterministic seed offset. The scenario generator stores data and metadata in explicit files, while benchmark settings are visible in function calls and constructor defaults. GPU timing includes synchronization, and path validation is separated from optimization so that generated trajectories are checked independently of the optimizer's objective.

There are also clear limits. Reproducibility is not the same as bitwise determinism: CUDA kernels can vary across hardware, software stacks, and libraries. The SciPy spline implementation introduces a CPU-side numerical component. In addition, the five-scenario dataset is not fully consumed by the planner loop, so the default execution is not a multi-scenario study.

14 Validation and Scientific Reliability

The validation procedure checks five conditions: numerical finiteness, two-dimensional shape with at least two points, start-point error below 10^{-6}, goal-point error below 10^{-6}, and complete containment inside the workspace. The combined Boolean validity flag requires all conditions simultaneously.

This validation is useful but deliberately narrow. It checks geometry and numerical integrity, not scientific optimality. A path may be finite, bounded, and endpoint-correct while still being long, poorly smoothed, or unsafe under a different collision interpretation. The source appropriately retains the separate collision-count and clearance metrics for this reason.

The audit-report generator additionally documents three source-level claims: stationary time rollout for metaheuristic evaluations, the squared-curvature-style smoothness term, and endpoint/boundary guarantees. These are useful documentation targets because they correspond directly to implementation choices.

15 Limitations and Technical Considerations

15.1 Scenario utilization

The clearest benchmark limitation is the gap between scenario generation and scenario execution. Five scenarios are generated, but only `scenario_1` is loaded into the environment used by all planners. A future multi-scenario study would need an outer loop over scenario metadata, with metrics aggregated across scenarios rather than only repeated stochastic runs on one environment.

15.2 Discrete versus continuous safety models

D* and ACO see obstacle occupancy through inflated cells, whereas continuous planners are evaluated using sampled path points and time-indexed obstacle geometry. These are compatible evaluation ideas but not identical state constraints. A high-resolution or continuous collision checker would provide a stronger geometric equivalence if that became a research requirement.

15.3 Penalty-domain issue

The clearance penalty formula has a denominator $c + 0.1$ whenever $c < 0.3$. Since c may be negative before reporting clamp, the denominator can approach zero or become negative. This is a genuine numerical property of the current code. It is better to identify it explicitly than to assume the objective is always nonnegative.

15.4 Fallback paths

When D* cannot find a route, the planner returns the direct start-to-goal path. That fallback keeps the return type stable, but it is not a proof of feasibility. The validation layer helps expose this distinction through collision and clearance measurements.

15.5 Incremental replanning scope

The D* Lite class contains the expected state variables and update logic, but `plan` clears its state before each solve. Consequently, the source does not demonstrate persistent map-change replanning across successive calls. The current benchmark compares fresh planning executions rather than a long-lived online replanning episode.

15.6 GPU vectorization boundary

The project should not be described as fully GPU-resident. Spline generation uses SciPy on CPU arrays, and several metaheuristic loops repeatedly transfer candidates between NumPy/SciPy and PyTorch. ACO also uses Python-level path construction and pheromone deposition loops. The appropriate engineering description is that the computationally expensive geometric cost kernel has been moved to the GPU where practical, while the control logic remains hybrid CPU/GPU.

16 Overall Technical Assessment

The implementation is coherent as a comparative navigation benchmark. Its strongest features are the centralized environment model, explicit spatiotemporal collision evaluation, consistent trajectory validation, reproducibility controls, multiple planner families, and structured export of machine-readable benchmark artifacts. The code also demonstrates awareness of practical GPU benchmarking concerns through CUDA synchronization, device-resident obstacle parameters, and tensorized distance calculations.

The most research-relevant architectural idea is the separation of planning search from path evaluation. This allows fundamentally different planners to be compared under the same geometric objective and validation rules. The D*-PSO hybrid is especially clear in this respect: a discrete global route is used as a guide for a continuous refinement stage rather than forcing either representation to perform all roles.

A rigorous reading should nevertheless avoid overclaiming. The benchmark's default execution covers one scenario, the GPU pipeline is heterogeneous rather than fully end-to-end vectorized, and some objective terms have numerical edge cases. These observations are not disqualifications; they are exactly the kind of implementation-level distinctions that matter when a computational project is presented as scientific software.

17 Conclusion

The program implements a complete, research-oriented software pipeline for two-dimensional robot navigation in the presence of moving pedestrian obstacles. It combines synthetic dynamic-environment generation, discrete occupancy reasoning, continuous spline trajectory construction, stochastic metaheuristic optimization, D* Lite graph search, GPU-accelerated cost evaluation, validation, benchmarking, and artifact packaging in one coherent program.

From a technical perspective, the project demonstrates skills relevant to scientific computing and research software, including structured object-oriented design, mathematical formulation of path-quality criteria, stochastic optimization, graph-based planning, GPU tensor programming, reproducibility controls, runtime instrumentation, and machine-readable result export. Its main methodological strength is consistency: once a planner produces a trajectory, the same environment-level machinery measures its geometric and spatiotemporal properties.

The main interpretive constraint is also clear. The default benchmark is a repeated comparison on one loaded scenario, not a five-scenario statistical study. The implementation should also be described as selectively GPU-accelerated rather than entirely GPU-native. With these qualifications, the project provides a solid foundation for a technical portfolio and a useful starting point for a broader multi-scenario robotics study.

18 Code-to-Report Traceability Appendix

The following table maps the major source-code blocks to the concepts discussed in this report. Line numbers refer to the supplied Python source.

Component	Lines	Role and report coverage
`PedestrianObstacle`	132–156	Data model and harmonic obstacle kinematics.
`generate_benchmark_dataset`	159–210	Scenario generation, deterministic seeding, CSV/JSON export.
`NavigationEnvironment`	213–377	Shared environment state, occupancy grid, GPU obstacle tensors, path metrics.
`generate_spline_path`	384–423	Cubic spline construction, fallback interpolation, clipping, exact endpoints.
`DStarPlanner`	430–597	D* Lite state, heuristic, priority queue, shortest-path computation, reconstruction.
`PSOPlanner`	604–714	Standard PSO, optional LDW inertia schedule, spline population evaluation.
`DStarPSOHybrid`	717–737	D* guide followed by LDW-PSO refinement.
`ABCPlanner`	744–869	Employed, onlooker, and scout phases.
`PSOABCHybrid`	872–985	PSO velocity dynamics plus differential swarm-colony exploration.
`SMOPlanner`	988–1103	Group-based local leader and global leader search.
`ACOPlanner`	1106–1224	Grid pheromone tensor, stochastic path construction, evaporation and deposit.
`validate_path`	1231–1252	Finite-value, shape, endpoint, and workspace checks.
Colab export helpers	1255–1276	Optional automatic download and notebook export.
`write_audit_report`	1279–1312	Writes the scientific audit summary from validation artifacts.
`package_submission`	1315–1337	Creates the zip submission archive.
`run_full_pipeline`	1344–1661	Benchmark orchestration, repeated runs, timing, memory, validation, exports, plotting.
Main execution	1669–1677	Invokes the benchmark, audit creation, packaging, and optional Colab download.

19 Terminology and Notation Reference

Symbol / term	Meaning in this report
W, H	Workspace width and height.
N_x, N_y	Number of occupancy-grid columns and rows.
K	Number of optimized via points.
$D = 2K$	Dimension of a continuous optimizer's control vector.
L	Sampled path length.
S	Discrete squared-curvature-style smoothness term.
c	Minimum signed clearance before reporting clamp.
C	Collision count over sampled point-obstacle pairs.
J	Shared scalar objective used by the continuous planners.
t_0	Deployment/start time for a path evaluation.
v	Assumed robot traversal speed in the time rollout.
$\|V\|, \|E\|$	Numbers of grid vertices and edges.
I, N, M	Generic iteration, population/particle, and ant counts in complexity discussions.